

Tema 4 — Estructuras de datos concurrentes y lock-free

Fuentes: Teóricas 04-Listas y 05-colas, Práctica clase-04 (listas y skip lists), Guía 4, Labo 3. **Ejercicio 4 del parcial.**

1. ¿Qué significa que una estructura concurrente sea "correcta"? (safety)

1.1 El problema

En secuencial, una operación es un instante: la especificación dice cómo está el objeto **antes** y **después**, y los estados intermedios no importan. En concurrente eso se rompe:

- Cada llamada **dura un intervalo**, que va desde la **invocación** hasta la **respuesta**.
- Los intervalos de threads distintos **se pisan**, así que una llamada puede ver efectos a medio hacer de otra.

Entonces hace falta decir qué resultados son "aceptables" cuando las llamadas se superponen. Para eso se miran **historias**: la secuencia de invocaciones y respuestas de una ejecución.

Vocabulario:

- **Orden de programa**: las llamadas de un mismo thread están en orden y nunca se superponen.
- **Orden de tiempo real**: A **precede** a B ($A \rightarrow B$) si la respuesta de A ocurre **antes** de la invocación de B. Si los intervalos se pisan, A y B son **concurrentes** y ninguna precede a la otra.
- **Especificación secuencial**: qué hace cada método si se ejecutara solo. Por ejemplo, en una cola:
`enq(z)` deja `q · z`; `deq()` sobre `a · q'` devuelve `a` y deja `q'`, y sobre la cola vacía tira `EmptyException`.

Notación de las líneas de tiempo de abajo: `[---op---`] es el intervalo de la llamada, y `*` marca un posible punto de linealización.

1.2 Consistencia secuencial (SC)

Existe **algún** orden secuencial de todas las llamadas que:

1. respeta el **orden de programa** de cada thread, y
2. cumple la especificación secuencial.

No mira el tiempo real entre threads distintos. Ejemplo de la teórica:

```
A: [--enq(x)--]           [--deq()→y--]
B:                   [--enq(y)--]
```

Es **SC**: el orden $B.\text{enq}(y) \rightarrow A.\text{enq}(x) \rightarrow A.\text{deq}(y)$ respeta el orden de programa de A y de B, y es una cola válida. Pero en la realidad $\text{enq}(x)$ **terminó antes** de que empezara $\text{enq}(y)$, y aun así sale primero y . SC lo permite porque $\text{enq}(x)$ y $\text{enq}(y)$ son de threads distintos.

SC no es composicional. Dos colas p y q , cada una SC, pueden formar un sistema que no lo es:

A: $p.\text{enq}(x)$ $q.\text{enq}(x)$ $p.\text{deq}() \rightarrow y$

B: $q.\text{enq}(y)$ $p.\text{enq}(y)$ $q.\text{deq}() \rightarrow x$

1. Como A desencola y de p , en p tiene que ir primero $p.\text{enq}(y)$ (de B) y después $p.\text{enq}(x)$ (de A).
2. Como B desencola x de q , en q tiene que ir primero $q.\text{enq}(x)$ (de A) y después $q.\text{enq}(y)$ (de B).
3. Por orden de programa: $p.\text{enq}(x) \rightarrow q.\text{enq}(x)$ en A, y $q.\text{enq}(y) \rightarrow p.\text{enq}(y)$ en B.
4. Juntando todo: $p.\text{enq}(x) \rightarrow q.\text{enq}(x) \rightarrow q.\text{enq}(y) \rightarrow p.\text{enq}(y) \rightarrow p.\text{enq}(x)$. Es un ciclo, así que no hay ningún orden global.

1.3 Linealizabilidad ★

Cada llamada parece ocurrir de golpe, en un instante (su punto de linealización, PL) que está dentro de su intervalo.

Dicho con historias: existe un orden secuencial de todas las llamadas que:

1. respeta el **orden de tiempo real**: si $A \rightarrow B$, A va antes que B, y
2. cumple la especificación secuencial.

Las dos definiciones son equivalentes. Ponerle un PL a cada llamada dentro de su intervalo **es** elegir ese orden: ordenás las llamadas por su PL.

Propiedades:

- **Respetar el tiempo real**, y por eso es la que se usa para estructuras de datos.
- **Es composicional**: si cada objeto es linealizable, el sistema entero lo es.
- **Toda historia linealizable es SC**, pero no al revés. El ejemplo de 1.2 es SC y **no** es linealizable: $\text{enq}(x)$ precede a $\text{enq}(y)$ en tiempo real, así que x tendría que salir primero.

En criollo: es como si cada operación, en algún momento entre que la llamaste y que te respondió, pasara de golpe, en un "clic". Si podés ubicar ese clic para todas las operaciones y el resultado tiene sentido como si hubieran pasado de a una, en el orden de los clics, la estructura es linealizable.

1.4 Ejemplos con historias

Linealizable, gracias a la superposición:

A: [-----enq(x)---*--]

B: [-*--enq(y)-----]

C: [--deq()→y--*--]

`enq(x)` y `enq(y)` se pisan, así que podés poner el PL de `enq(y)` **antes** que el de `enq(x)`. El orden por PL es `enq(y)`, `enq(x)`, `deq()→y`, que es una cola válida.

No linealizable:

```
A: [--enq(x)-*-]
B:           [-*-enq(y)--]
C:                   [--deq()→y--]
```

Ahora `enq(x)` **termina** antes de que empiece `enq(y)`. Pongas donde pongas los PL, el de `enq(x)` queda antes que el de `enq(y)`, así que `deq` tendría que devolver `x`.

Registro:

```
A: [-----write(7)-----]
B:           [--read()→7--]
C:                   [--read()→0--]      (el registro arrancaba en 0)
```

- La lectura de B es válida, porque se pisa con la escritura: el PL de `write(7)` puede ir antes del de `read()→7`.
- La lectura de C **no**: empieza después de que `write(7)` terminó, así que tiene que ver 7 (o algo escrito después).

La trampa más común: leer varias variables sin atomicidad ("lectura rota").

```
// ajustarPosicion escribe x y después y, bajo un lock.
// getPosicion lee sin lock:
int px = x;           // lee el x NUEVO
int py = y;           // lee el y VIEJO (todavía no se escribió)
return (px, py);      // un par que NUNCA existió en el objeto
```

No existe ningún instante en que el objeto haya valido `(x nuevo, y viejo)`, así que no hay dónde poner el PL: **no es linealizable**. Por eso, para leer varios campos juntos hace falta el lock, un objeto inmutable detrás de una `AtomicReference`, o un double collect (sección 6).

1.5 Cómo encontrar el PL (receta)

1. **Escribí la función de abstracción:** qué valor abstracto representa el estado concreto. Por ejemplo: "un elemento está en el conjunto si y solo si está en un nodo alcanzable desde `head`, distinto de los centinelas y **no marcado**".
2. **Para las operaciones que modifican:** el PL es el paso atómico (una escritura, un CAS exitoso, o algo dentro del lock) en el que **cambia el valor abstracto**. Mirá la función de abstracción: ¿qué línea hace que el elemento "entre" o "salga"?
3. **Para las operaciones que solo leen, o que fallan:** el PL es el paso en el que se lee lo que **decide el resultado**. Por ejemplo, la lectura que ve el nodo sin marcar, o la que ve una clave mayor.
4. **Separá por casos:** exitosa y no exitosa pueden tener PL distintos, y hay que dar los dos.

5. Verificá tres cosas:

- el PL está **dentro** del intervalo de la llamada;
- es **atómico**: una sola instrucción, o un momento con el lock tomado;
- en ese instante, el valor abstracto cambia exactamente como dice la especificación (o, para una lectura, el valor leído coincide con el valor abstracto de ese instante).

Guía rápida:

- Con locks: cualquier instante dentro de la sección crítica. Lo típico es la escritura que modifica.
- Sin locks: el **CAS exitoso** que cambia el valor abstracto, o la lectura que decide el resultado.

El PL puede no ser un paso propio. Es el caso del `contains lazy` que falla (sección 4.4): a veces el PL cae en un paso de *otro* thread, o en un instante elegido según lo que pasó alrededor. Lo que importa es que exista un instante dentro del intervalo en el que el resultado sea correcto.

Plantilla para el parcial:

La función de abstracción es *op1* exitosa se linealiza en `<Línea>`, porque ahí `<qué cambia en el valor abstracto>`, y esa instrucción es atómica (`<CAS / escritura con el lock tomado>`). *op1* no exitosa se linealiza en `<Línea>`, donde lee `<lo que decide>`. *op2* Como cada PL está dentro del intervalo de su llamada y, ordenando las llamadas por PL, se obtiene una ejecución válida de `<especificación secuencial>`, la implementación es linealizable.

2. ¿Qué garantiza que avance? (progreso)

Garantía	Qué asegura	Tipo
Wait-free	toda llamada termina en una cantidad finita de pasos, haga lo que haga el resto	no bloqueante (la mejor)
Lock-free	alguna llamada siempre termina (progreso global; alguien puede sufrir inanición)	no bloqueante
Starvation-free	toda llamada termina, <i>si el scheduler es fair</i>	bloqueante (con locks)
Deadlock-free	alguna llamada termina, <i>si el scheduler es fair</i>	bloqueante

Wait-free $\Rightarrow$ lock-free. Con locks, lo mejor que podés tener es starvation-free, y eso **solo si los locks son fair**.

Cómo se justifica cada una:

- **Wait-free**: mostrás una cota de pasos que no depende de los demás. Por ejemplo, una sola lectura (`get()` de un `AtomicInteger`), o recorrer una lista sin reintentar nunca (`contains lazy` y lock-free).
- **Lock-free**: mostrás que **si mi CAS falla, es porque el CAS de otro tuvo éxito**, o sea que alguien progresó. Los reintentos son "culpa" de un éxito ajeno.

- **Deadlock-free con locks:** todos toman los locks en el **mismo orden total** (por ejemplo, head → tail, o índice creciente), así que no hay espera circular.
- **Starvation-free con locks:** deadlock-free, más locks fair, más ningún reintento que dependa de lo que hacen los demás.
- **Por qué el optimista y el lazy no son starvation-free** aunque los locks sean fair: el que reintenta porque la validación falló puede fallar **siempre**, si otros modifican justo ahí.

3. CAS y referencias atómicas: las herramientas del lock-free

`compareAndSet(esperado, nuevo)` : "si vale `esperado` , ponelo `nuevo` ", todo atómico. Devuelve si lo logró.

Patrón **CAS loop**:

```
AtomicInteger x = new AtomicInteger(0);

void inc() {
    while (true) {
        int v = x.get();
        if (x.compareAndSet(v, v + 1)) return;    // PL: el CAS exitoso
    }
}
```

- Es **lock-free**: si tu CAS falló, es porque **otro** cambió `x` , o sea que tuvo éxito.
- **No es wait-free**: te pueden ganar infinitas veces.

Varios campos que cambian juntos (la Figura, guía 4 ej 11): se meten en un objeto **immutable**, y se hace CAS sobre la referencia al objeto entero.

```
final class Estado { final int x, y; Estado(int x, int y) { this.x = x; this.y = y; } }
AtomicReference<Estado> pos = new AtomicReference<>(new Estado(0, 0));

void mover(int nx, int ny) {
    while (true) {
        Estado viejo = pos.get();
        if (pos.compareAndSet(viejo, new Estado(nx, ny))) return;    // PL
    }
}

Estado leer() { return pos.get(); }    // wait-free; nunca ve un (x, y) mezclado
```

Las otras referencias atómicas de la materia:

Clase	Qué agrupa	Para qué
<code>AtomicReference<T></code>	una referencia	CAS de un puntero (pila, cola, objeto immutable)

Clase	Qué agrupa	Para qué
<code>AtomicMarkableReference<T></code>	referencia + un bit	lista lock-free: <code>next</code> y <code>marked</code> como una sola unidad
<code>AtomicStampedReference<T></code>	referencia + un entero (sello)	evitar ABA: el sello cambia en cada modificación

```
// AtomicMarkableReference
boolean[] marca = new boolean[1];
Node succ = curr.next.get(marca); // lee referencia y marca juntas
curr.next.compareAndSet(succ, succ, false, true); // marca sin cambiar la referencia

// AtomicStampedReference
int[] sello = new int[1];
Node viejo = top.get(sello);
top.compareAndSet(viejo, nuevo, sello[0], sello[0] + 1); // falla si cambió el sello
```

4. Conjuntos con listas enlazadas (teórica 4 + labo 3) ★

La lista está ordenada por clave (el hash), con dos centinelas: `head` = $-\infty$ y `tail` = $+\infty$. Para modificar necesitás `pred` (el nodo anterior) y `curr` (el nodo en cuestión).

Función de abstracción: un elemento está en el conjunto si y solo si está en un nodo **alcanzable desde head**, distinto de los centinelas. En la lazy y la lock-free se agrega: **y no está marcado**.

Versión	Idea en criollo	contains	Progreso
Gruesa	un solo candado para toda la lista (<code>synchronized</code>)	con lock	deadlock-free; sin inanición si el lock es fair
Fina (hand-over-hand)	cada nodo tiene su candado; avanzás "de mano en mano": tomás el siguiente antes de soltar el anterior. Siempre tenés tomados <code>pred</code> y <code>curr</code>	con locks	deadlock-free (todos toman en orden <code>head</code> → <code>tail</code>); sin inanición si los locks son fair
Optimista	recorrés sin locks; tomás <code>pred</code> y <code>curr</code> ; validás recorriendo de nuevo desde <code>head</code> que <code>pred</code> sigue siendo alcanzable y que <code>pred.next == curr</code> ; si no, reintentás	con locks + validación	deadlock-free; puede haber inanición aunque los locks sean fair (siempre te cambian la lista)
Lazy	cada nodo tiene <code>marked</code> . Borrar = primero marcar (borrado lógico) y después desenganchar. <code>validate = !pred.marked && !curr.marked && pred.next == curr</code> , sin recorrer de nuevo	wait-free , sin locks	add/remove pueden sufrir inanición
Lock-free	sin locks. <code>next</code> y <code>marked</code> son una sola unidad (<code>AtomicMarkableReference</code>), así que el CAS falla si el	wait-free	add/remove lock-free

Versión	Idea en criollo	contains	Progreso
	nodo está marcado. <code>find()</code> va limpiando los nodos marcados que encuentra en el camino		

4.1 Tabla de PL por versión ☆

Versión	add / remove exitosos	add / remove que fallan	contains
Gruesa	la escritura <code>pred.next = ...</code>	cualquier momento con el lock tomado	cualquier momento con el lock tomado
Fina	la escritura <code>pred.next = ...</code>	cuando toma el lock de un nodo con clave $\geq$ la buscada	ídem
Optimista	la escritura <code>pred.next = ...</code>	cuando tiene los locks y la validación da bien	ídem (con validación exitosa)
Lazy	add: <code>pred.next = node</code> ; remove: <code>curr.marked = true</code>	como en la optimista	encuentra: cuando ve el nodo sin marcar ; no encuentra: ver 4.4
Lock-free	add: el CAS que engancha el nodo; remove: el CAS que marca	cuando <code>find</code> devuelve la ventana y se evalúa la clave	encuentra: lee el nodo y ve la marca en <code>false</code> ; no encuentra: lee una clave mayor, o ve el nodo marcado

Por qué el remove lazy y el lock-free se linealizan al marcar y no al desenganchar: la función de abstracción dice "alcanzable **y no marcado**". Desde que se marca, el elemento ya no está en el conjunto, aunque el nodo siga enganchado (y `contains` devuelve `false`). Desenganchar después es limpieza, que no cambia el valor abstracto.

Por qué en la optimista el PL exige validación exitosa: si la validación falla, la llamada no "pasó". Reintenta, y el PL es el del intento que sí valida.

4.2 Granularidad fina (hand-over-hand)

```
public boolean remove(T item) {
    int key = item.hashCode();
    head.lock();
    Node pred = head;
    try {
        Node curr = pred.next;
        curr.lock();
        try {
            while (curr.key < key) {
                pred.unlock();           // suelto el de atrás...
                pred = curr;
                curr = curr.next;
                curr.lock();             // ...y tomo el de adelante
            }
        }
    }
}
```

```

    }
    if (curr.key == key) {
        pred.next = curr.next; // PL (exitoso)
        return true;
    }
    return false;
} finally { curr.unlock(); }
} finally { pred.unlock(); }
}

```

- **¿Por qué hay que tomar también el lock del nodo que borro?** Si solo tomás el de `pred`, otro thread puede estar borrando `curr` al mismo tiempo (con `curr` como su `pred`), y uno de los dos borrados se pierde. Ejemplo: `a → b → c`; A borra `b` (tiene `a`) y B borra `c` (tiene `b`). A hace `a.next = c` y B hace `b.next = d`, y `c` sigue en la lista.
- **Orden de los locks:** siempre `pred` antes que `curr`, de head hacia tail. Si en algún lado cambiás el orden → deadlock (guía 4 ej 4b).

4.3 Optimista

```

private boolean validate(Node pred, Node curr) {
    Node aux = head;
    while (aux.key <= pred.key) {
        if (aux == pred) return pred.next == curr; // pred alcanzable y adyacente a curr
        aux = aux.next;
    }
    return false;
}

public boolean remove(T item) {
    int key = item.hashCode();
    while (true) {
        Node pred = head, curr = pred.next;
        while (curr.key < key) { pred = curr; curr = curr.next; } // sin locks
        pred.lock();
        try {
            curr.lock();
            try {
                if (validate(pred, curr)) {
                    if (curr.key == key) { pred.next = curr.next; return true; } // PL
                    return false;
                }
            } finally { curr.unlock(); }
        } finally { pred.unlock(); }
        // si no validó: reintenta desde head
    }
}

```

```
}  
}
```

- Recorrer sin locks es seguro porque un nodo desenganchado **no cambia su** `next`, y el garbage collector no lo recicla mientras alguien lo mira. Una cadena de nodos huérfanos eventualmente vuelve a la lista.
- `contains` también toma locks y valida.
- Para el ej 4a de la guía: la inanición aparece cuando **siempre** te cambian la lista entre el recorrido y la validación.

4.4 Lazy

```
private boolean validate(Node pred, Node curr) {  
    return !pred.marked && !curr.marked && pred.next == curr;    // sin recorrer  
}  
  
// remove: igual que la optimista, pero adentro del validate exitoso:  
if (curr.key == key) {  
    curr.marked = true;           // PL: borrado lógico  
    pred.next = curr.next;       // borrado físico  
    return true;  
}  
  
public boolean contains(T item) {    // wait-free: sin locks, sin reintentos  
    int key = item.hashCode();  
    Node curr = head;  
    while (curr.key < key) curr = curr.next;  
    return curr.key == key && !curr.marked;  
}
```

El PL del `contains` que no encuentra (sutil):

1. A busca `a` y, recorriendo, cae en un nodo `a` que **ya está marcado** (borrado lógicamente, quizás hasta desenganchado).
2. Si nadie agrega `a` mientras tanto, el PL es cuando A lee la marca: en ese instante `a` no está en el conjunto.
3. Pero mientras A estaba en el nodo viejo, otro thread pudo hacer `add(a)` y enganchar un nodo `a` **nuevo** en la parte alcanzable. Cuando A lee la marca, `a` **sí** está en el conjunto, así que ese instante no sirve.
4. Solución: el PL es **justo antes** de que el `add(a)` concurrente enganche su nodo. Ese instante está dentro del intervalo de A, porque A ya había llegado al nodo marcado, y en ese momento `a` no estaba.

O sea, **el PL puede ser un instante determinado por el orden global de eventos, no un paso del propio thread.**

4.5 Lock-free

```
public Window find(Node head, int key) {           // devuelve (pred, curr) con curr.key >= key
    boolean[] marked = {false};
    retry:
    while (true) {
        Node pred = head, curr = pred.next.getReference();
        while (true) {
            Node succ = curr.next.get(marked);
            while (marked[0]) {                     // curr está marcado: lo desengancho
                if (!pred.next.compareAndSet(curr, succ, false, false)) continue retry;
                curr = succ;
                succ = curr.next.get(marked);
            }
            if (curr.key >= key) return new Window(pred, curr);
            pred = curr;
            curr = succ;
        }
    }
}

public boolean add(T item) {
    int key = item.hashCode();
    while (true) {
        Window w = find(head, key);
        if (w.curr.key == key) return false;        // PL (falla)
        Node node = new Node(item);
        node.next = new AtomicMarkableReference<>(w.curr, false);
        if (w.pred.next.compareAndSet(w.curr, node, false, false)) return true;    // PL (exitoso)
    }
}

public boolean remove(T item) {
    int key = item.hashCode();
    while (true) {
        Window w = find(head, key);
        if (w.curr.key != key) return false;        // PL (falla)
        Node succ = w.curr.next.getReference();
        if (!w.curr.next.compareAndSet(succ, succ, false, true)) continue;    // PL (exitoso): marcar
        w.pred.next.compareAndSet(w.curr, succ, false, false);                // limpieza (puede
fallar, no importa)
        return true;
    }
}
```

- **¿Por qué `next` y `marked` tienen que ser una sola unidad?** Si fueran campos separados: A marca `c` y B, que no lo vio marcado, hace CAS sobre `c.next` para borrar `d`, y engancha algo a un nodo que ya salió de la lista. Con `AtomicMarkableReference`, el CAS de B **falla** porque la marca ya no es `false`.
- **Lock-free:** un thread reintenta solo si falló un CAS, y eso pasa porque otro modificó ese enlace con éxito.
- `contains` es igual al de la lazy, pero leyendo `curr.next.isMarked()`, y es **wait-free**.

Skip lists (práctica 4, prioridad baja)

- Una lista con varios niveles de "atajos" que simula un árbol. Búsqueda esperada $O(\log n)$, sin las rotaciones de un AVL (que generan contención).
- El nivel de cada nodo es aleatorio, con probabilidad p^i .
- Hay versión **Lazy** (locks + `marked` + `fullyLinked`) y versión **LockFree**. En las dos, `contains` es wait-free.

5. Colas y pilas (teórica 5)

5.1 Cola acotada con dos locks

`enqLock` protege el tail y `deqLock` el head, así que un productor y un consumidor pueden trabajar a la vez. `size` es un `AtomicInteger`, porque lo tocan los dos bandos sin compartir lock.

```
public void enq(T x) {
    boolean despertarConsumidores = false;
    Node e = new Node(x);
    enqLock.lock();
    try {
        while (size.get() == capacity) notFullCondition.await();
        tail.next = e;           // PL
        tail = e;
        if (size.getAndIncrement() == 0) despertarConsumidores = true; // estaba vacía
    } finally { enqLock.unlock(); }
    if (despertarConsumidores) {
        deqLock.lock();           // tomo el lock del OTRO bando para avisar
        try { notEmptyCondition.signalAll(); } finally { deqLock.unlock(); }
    }
}

public T deq() {
    T result;
    boolean despertarProductores = false;
    deqLock.lock();
    try {
        while (head.next == null) notEmptyCondition.await();

```

```

        result = head.next.value;
        head = head.next;                // PL
        if (size.getAndDecrement() == capacity) despertarProductores = true;    // estaba llena
    } finally { deqLock.unlock(); }
    if (despertarProductores) {
        enqLock.lock();
        try { notFullCondition.signalAll(); } finally { enqLock.unlock(); }
    }
    return result;
}

```

- **Solo se despierta al otro bando en las transiciones** "estaba vacía" y "estaba llena".
- **Por qué no se pierde el wake-up:** el productor ve la cola llena en dos pasos (lee `size` y después hace `await`). Como el consumidor, para avisar, **tiene que tomar** `enqLock`, su `signal` no puede caer en el medio de esos dos pasos.
- **PL del `enq` = `tail.next = e`, no `tail = e`:** desde que se engancha, el elemento ya es parte de la cola (un consumidor lo puede sacar), aunque `tail` todavía no avanzó.
- **`size` puede quedar en `-1` un instante:** con la cola vacía, el productor engancha `e` y se duerme antes de incrementar `size`. Un consumidor ve `head.next != null`, lo saca y decrementa: `-1`. Cuando el productor sigue, vuelve a 0. El consumidor no mira `size` para decidir si hay elementos (mira `head.next`), así que no rompe nada.

5.2 Cola no acotada total

- `enq` siempre tiene éxito; `deq` sobre la cola vacía **tira** `EmptyException` en vez de esperar.
- PL: `enq` en `tail.next = e`; `deq` exitoso en `head = head.next`; `deq` que falla cuando lee `head.next == null`.
- Cada método toma un solo lock → deadlock-free, y starvation-free si los locks son fair.

5.3 Cola lock-free (Michael–Scott)

```

public void enq(T value) {
    Node node = new Node(value);
    while (true) {
        Node last = tail.get();
        Node next = last.next.get();
        if (last == tail.get()) {                // nadie movió tail mientras leía
            if (next == null) {
                if (last.next.compareAndSet(null, node)) {    // PL
                    tail.compareAndSet(last, node);           // si falla, alguien ya lo ayudó
                    return;
                }
            } else {
                tail.compareAndSet(last, next);                // helping: tail atrasado, lo avanzo yo
            }
        }
    }
}

```

```

    }
}

public T deq() throws EmptyException {
    while (true) {
        Node first = head.get(), last = tail.get();
        Node next = first.next.get();
        if (first == head.get()) {
            if (first == last) {
                if (next == null) throw new EmptyException(); // PL (vacía): la lectura de next
                tail.compareAndSet(last, next); // helping
            } else {
                T value = next.value;
                if (head.compareAndSet(first, next)) return value; // PL (exitoso)
            }
        }
    }
}
}

```

- **El enq son dos pasos:** enganchar (`last.next`) y avanzar `tail` . Entre los dos, `tail` queda **atrasado**.
- **Helping:** cualquiera que ve `tail` atrasado lo avanza. Así nadie queda trabado esperando a un productor lento, y eso hace que la cola sea lock-free.
- **Lock-free:** un CAS falla solo si otro thread modificó ese puntero con éxito.

5.4 Pila lock-free

```

protected boolean tryPush(Node node) {
    Node oldTop = top.get();
    node.next = oldTop;
    return top.compareAndSet(oldTop, node); // PL (exitoso)
}

protected Node tryPop() throws EmptyException {
    Node oldTop = top.get();
    if (oldTop == null) throw new EmptyException(); // PL (vacía)
    Node newTop = oldTop.next;
    return top.compareAndSet(oldTop, newTop) ? oldTop : null; // PL (exitoso)
}

public void push(T value) {
    Node node = new Node(value);
    while (!tryPush(node)) backoff.backoff(); // falló: espero un rato y reintento
}

```

- **Backoff exponencial:** cuando el CAS falla, hay contención. Esperar un tiempo aleatorio, que crece con cada fallo, baja los choques.
- Todo pasa por `top`: es un cuello de botella, y por eso existe la **eliminación** (5.6).

5.5 Problema ABA ★ (en C, o si reciclás nodos a mano sin garbage collector)

1. El thread 1 empieza un `pop`: lee `top = A` y `next = B`, y se duerme antes del CAS.
2. El thread 2 hace `pop` (saca A) y `pop` (saca B), y después `push` de un nodo **reciclado**, que resulta ser el mismo A.
3. El thread 1 despierta: su CAS "¿`top` sigue siendo A?" da **verdadero** y pone `top = B`, un nodo que ya no está en la pila.

Solución: `AtomicStampedReference`, con un sello (versión) que se incrementa en cada cambio. Aunque la dirección vuelva a ser la misma A, el sello ya no coincide y el CAS falla.

```
int[] sello = new int[1];
Node oldTop = top.get(sello);
Node newTop = oldTop.next;
if (top.compareAndSet(oldTop, newTop, sello[0], sello[0] + 1)) { ... }
```

En Java con garbage collector no pasa, porque un nodo no se recicla mientras alguien tenga una referencia a él.

5.6 Otras (prioridad baja)

- **Eliminación:** un push y un pop que se cruzan se cancelan entre sí sin tocar la pila (con un `Exchanger`, en un arreglo de intercambiadores).
- **Cola síncrona / rendezvous:** el que pone espera a que alguien lo saque. Estructuras duales.

6. Recetas para el Ej 4 del parcial

6.1 Granularidad fina sobre algo con N partes

- Un lock por parte.
- Las operaciones sobre varias partes **toman los locks en orden creciente**, así que no hay deadlock.
- Las operaciones sobre "todo" (una suma) toman todos los locks en orden. Mientras los tienen todos hay un snapshot real, y el PL puede ser cualquier momento con **todos** tomados.

```
Lock[] locks;          // uno por posición
Object[] recursos;

void swap(int i, int j) {
    int a = Math.min(i, j), b = Math.max(i, j);    // orden global: menor índice primero
    locks[a].lock();
    locks[b].lock();
```

```

    try { Object aux = recursos[i]; recursos[i] = recursos[j]; recursos[j] = aux; }    // PL: con los
dos tomados
    finally { locks[b].unlock(); locks[a].unlock(); }
}

```

Sin inanición solo si los locks son fair (guía 4 ej 3b).

6.2 Optimista (guía 4 ej 5)

Leer, computar **sin** lock, tomar el lock, **validar** que lo que leíste sigue igual (con un número de versión), y escribir o reintentar.

```

while (true) {
    int v; Object viejo;
    lock.lock(); try { viejo = tabla.get(i); v = version[i]; } finally { lock.unlock(); }
    Object nuevo = computacionalmenteCostoso(viejo);          // sin lock
    lock.lock();
    try {
        if (version[i] == v) { tabla.put(i, nuevo); version[i]++; return; }    // PL
    } finally { lock.unlock(); }
    // alguien lo cambió mientras computaba: reintento
}

```

Sin inanición: **no**, porque te pueden cambiar la entrada siempre.

6.3 Lock-free con CAS

- El CAS loop (sección 3).
- Para modificar varios campos juntos: un objeto **immutable** en una `AtomicReference`, y CAS del objeto entero (Figura, guía 4 ej 11).
- Para clasificar el progreso: una sola lectura o una sola escritura atómica es **wait-free**; un CAS loop es **lock-free** (guía 4 ej 10).

6.4 Double collect (leer todo dos veces y comparar)

```

int[] collect() { int[] c = new int[n]; for (int i = 0; i < n; i++) c[i] = valor[i].get(); return c;
}

int get() {
    int[] antes = collect();
    while (true) {
        int[] despues = collect();
        if (Arrays.equals(antes, despues)) return suma(despues);    // PL: entre los dos collect
        antes = despues;
    }
}

```

```
}  
}
```

- **Por qué es linealizable:** si ninguna celda cambió entre sus dos lecturas, entonces en cualquier instante entre el fin del primer collect y el principio del segundo **todas las celdas valían eso a la vez**. Ese instante es el PL.
- **Con valores que solo crecen**, que las dos lecturas coincidan implica que la celda no cambió. Y alcanza con comparar **las sumas**: como ninguna celda puede bajar, si la suma de las dos pasadas es igual, ninguna celda cambió (así lo resuelve el ej 4 del simulacro, con solo `inc`).
- **Con decrementos o `reset` no alcanza:**
 - comparar sumas falla, porque dos celdas pueden cambiar compensándose (+1 en una, -1 en otra);
 - comparar arreglo contra arreglo también falla, porque una celda puede cambiar y **volver** al mismo valor (ABA), y entonces no hubo snapshot;
 - hacen falta versiones por celda (un contador de cambios que solo crece).
- **No es wait-free:** si siempre hay escrituras, puede repetir infinitamente. Es lock-free si "progreso" incluye que las escrituras terminen.

6.5 Caso fino: la suma con un solo collect también puede ser linealizable

En un contador de N celdas donde **solo** hay `inc` de a 1, un `get()` que suma leyendo cada celda **una vez** (sin locks) **es linealizable**:

- Cada celda se lee con un valor $\geq$ el que tenía al empezar el `get` y $\leq$ el que tiene al terminar, porque solo crecen. Entonces la suma leída `v` cumple `total_inicio ≤ v ≤ total_fin`.
- El total real sube **de a 1**, así que pasa por **todos** los enteros entre `total_inicio` y `total_fin`. En particular, hay un instante dentro del intervalo del `get` en que el total real vale exactamente `v`. Ese es el PL.

Con decrementos, `reset`, o incrementos de más de 1, este argumento se cae, y hace falta double collect con versiones o tomar todos los locks. La solución oficial del simulacro usa este mismo argumento en su inciso extra (`get` con locks e `inc` con CAS).

7. Guía 4 — qué patrón entrena cada ejercicio

Ej	Patrón
1	listas con claves repetidas: qué cambia en cada versión
2	Figura: un lock por grupo de campos + orden fijo si tocás los dos
3	Recursos <code>swap(i, j)</code> : lock por posición, tomar min primero; ¿inanición?
4	optimista: inanición (a) y orden de locks invertido (b)
5	actualizarEntrada optimista (computar fuera del lock + validar)
6	cola con dos pilas: la race de <code>deq</code> y cómo arreglarla

Ej	Patrón
7	cola acotada con dos contadores (menos contención sobre <code>size</code>)
8	pila acotada
9	ABA en la pila lock-free + sellos
10	contador lock-free inc / get / reset: ¿cuáles son wait-free?
11	Figura lock-free (objeto inmutable + CAS)